

# TypeScript Programming Foundations

---

## 5-Day Beginner TypeScript Course

## Table of Contents

---

1. [Course Overview](#)
2. [Day 1: TypeScript Basics - Types, Variables & Functions](#)
3. [Day 2: Advanced Types & Type Systems](#)
4. [Day 3: Classes, Interfaces & Object-Oriented Programming](#)
5. [Day 4: Generics, Utility Types & Advanced Patterns](#)
6. [Day 5: TypeScript Project & Real-World Applications](#)
7. [Final Assessment](#)
8. [TypeScript Quick Reference](#)
9. [Glossary](#)

## Course Overview

---

### What You Will Learn

By the end of this 5-day TypeScript course, you will be able to:

- Understand TypeScript's type system and how it extends JavaScript
- Write type-safe code with primitive, object, and union types
- Use interfaces and type aliases for reusable type definitions
- Implement classes with access modifiers (public, private, protected)
- Use generics to create flexible, reusable components
- Leverage utility types for advanced type transformations
- Configure a TypeScript project with tsconfig.json
- Build real-world applications with TypeScript

### Prerequisites

- Basic understanding of JavaScript (variables, functions, arrays, objects)
- Node.js installed (version 14+)
- VS Code or any code editor with TypeScript support

### How to Set Up TypeScript

```
# Install TypeScript globally
npm install -g typescript

# Check installation
tsc --version

# Create a new project
mkdir typescript-course
cd typescript-course
npm init -y

# Install TypeScript as dev dependency
npm install typescript --save-dev

# Initialize TypeScript configuration
npx tsc --init

# Compile TypeScript to JavaScript
npx tsc

# Run in watch mode (auto-compile)
npx tsc --watch
```

# Day 1: TypeScript Basics

---

## Types, Variables & Functions

---

### Learning Objectives

By the end of Day 1, you will be able to:

- Understand TypeScript's basic types
- Annotate variables with types
- Work with arrays and tuples
- Define and use functions with typed parameters and return values
- Use the TypeScript compiler (tsc)

## 1.1 What is TypeScript?

---

### TypeScript is JavaScript with Types

TypeScript is a superset of JavaScript that adds static typing. Every valid JavaScript code is also valid TypeScript code.

### Key Benefits:

- **Type Safety:** Catch errors during development, not at runtime
- **Better IDE Support:** Autocompletion, inline documentation, refactoring
- **Self-Documenting:** Types serve as documentation
- **Modern JavaScript:** Access to latest ES features before they're fully supported

## TypeScript vs JavaScript

| JavaScript               | TypeScript                          |
|--------------------------|-------------------------------------|
| Dynamic types            | Static types (with optional typing) |
| Errors caught at runtime | Errors caught at compile time       |
| No type checking         | Type checking with tsc              |
| No interfaces            | Interfaces and type aliases         |
| No generics              | Generics for reusable components    |

## How TypeScript Works

TypeScript Source (.ts) → TypeScript Compiler (tsc) → JavaScript (.js)

## 1.2 Basic Types

### 1. Primitive Types

| Type      | Description                  | Example         |
|-----------|------------------------------|-----------------|
| string    | Text data                    | "Hello"         |
| number    | All numbers (integer, float) | 25 , 3.14 , -10 |
| boolean   | True or false                | true , false    |
| null      | Null value                   | null            |
| undefined | Undefined value              | undefined       |
| symbol    | Unique identifier            | Symbol("id")    |

## Examples:

```
// String
let name: string = "Alice";
let greeting: string = "Hello, " + name;

// Number
let age: number = 25;
let price: number = 19.99;
let temperature: number = -5;

// Boolean
let isStudent: boolean = true;
let isActive: boolean = false;

// Null & Undefined
let empty: null = null;
let notDefined: undefined = undefined;
```

## 2. BigInt

For very large numbers:

```
let largeNumber: bigint = 9007199254740991n;
let anotherLarge: bigint = 1000000000000000000n;
```

## 3. Literal Types

Specific values as types:

```
let status: "success" | "error" | "pending" = "success";
let direction: "left" | "right" = "left";
let answer: 42 | "answer" = 42;
```

# 1.3 Type Annotations

---

## Explicit Type Annotations

```
// Variable annotation: variableName: type = value
let name: string = "John";
let age: number = 30;
let isEmployed: boolean = true;
let scores: number[] = [85, 90, 78];
```

```
// Function annotation
function greet(name: string): string {
    return "Hello, " + name;
}

// Arrow function annotation
const add = (a: number, b: number): number => a + b;
```

## Type Inference

TypeScript can infer types automatically:

```
// TypeScript infers the type (no annotation needed)
let message = "Hello"; // Type: string
let count = 5;         // Type: number
let flag = true;        // Type: boolean

// Function return type inference
function multiply(a: number, b: number) {
    return a * b; // TypeScript infers return type: number
}
```

## When to Use Type Annotations

```
// ✓ Use annotations for function parameters
function processUser(user: User): void { ... }

// ✓ Use annotations for variables that might be ambiguous
let value: string | number = "hello"; // Could be string or number

// ✓ Use annotations when you want to be explicit about the type
let apiKey: string = "abc123";

// ✗ Not necessary when TypeScript can infer the type
let name = "Alice"; // TypeScript knows it's a string
```

## 1.4 Arrays and Tuples

---

### Arrays

```
// Array of numbers
let numbers: number[] = [1, 2, 3, 4, 5];
let moreNumbers: Array<number> = [6, 7, 8]; // Alternative syntax
```

```
// Array of strings
let fruits: string[] = ["apple", "banana", "cherry"];

// Array of mixed types (union)
let mixed: (string | number)[] = ["hello", 42, "world", 100];

// Empty array with type annotation
let emptyList: number[] = [];

// Array operations (type-safe)
let scores: number[] = [85, 90, 78];
scores.push(92); // Valid
// scores.push("85"); // Error: Argument of type 'string' is not assignable to parameter of type 'number'.
```

## Tuples

Tuples are arrays with a fixed number of elements, each with a specific type:

```
// Tuple declaration
let person: [string, number] = ["Alice", 25];

// Access elements
let name: string = person[0]; // "Alice"
let age: number = person[1];  // 25

// Tuple with optional elements
let optionalTuple: [string, number?] = ["Alice"]; // valid
optionalTuple = ["Alice", 25]; // also valid

// Tuple with rest elements
let rgb: [number, number, number] = [255, 128, 0];
let header: [string, ...number[]] = ["Header", 1, 2, 3, 4];
```

## 1.5 Functions

### Function Parameter Types

```
// Basic function with typed parameters
function greet(name: string): string {
    return "Hello, " + name;
}

// Multiple parameters
function calculateRectangle(width: number, height: number): number {
    return width * height;
}
```

```
}

// Optional parameters (with ?)
function buildName(firstName: string, lastName?: string): string {
    if (lastName) {
        return firstName + " " + lastName;
    }
    return firstName;
}

// Default parameters
function greetWithDefault(name: string = "Guest"): string {
    return "Hello, " + name;
}

console.log(greetWithDefault()); // "Hello, Guest"
console.log(greetWithDefault("Alice")); // "Hello, Alice"
```

## Rest Parameters

```
// Rest parameters (any number of arguments)
function sumAll(...numbers: number[]): number {
    return numbers.reduce((total, num) => total + num, 0);
}

console.log(sumAll(1, 2, 3, 4, 5)); // 15
console.log(sumAll(10, 20)); // 30
```

## Function Return Types

```
// Explicit return type
function add(a: number, b: number): number {
    return a + b;
}

// Void return (no return value)
function logMessage(message: string): void {
    console.log(message);
    // No return statement, or return with no value
}

// Never return (function never completes)
function throwError(message: string): never {
    throw new Error(message);
}

function infiniteLoop(): never {
    while (true) {
        // never returns
    }
}
```

```
}  
}
```

## Arrow Functions

```
// Arrow function with explicit type  
const multiply = (a: number, b: number): number => a * b;  
  
// Arrow function with inferred type  
const divide = (a: number, b: number) => a / b;  
  
// Arrow function with object return  
const createPerson = (name: string, age: number): { name: string; age: number } => ({  
    name,  
    age  
});
```

## Function Types

```
// Declaring a variable as a function type  
let operation: (a: number, b: number) => number;  
  
// Assign functions to the variable  
operation = add;  
operation = multiply;  
  
// Using the function type variable  
console.log(operation(5, 3)); // 8 (if add) or 15 (if multiply)  
  
// Function type as parameter  
function processOperation(callback: (a: number, b: number) => number): number {  
    return callback(10, 5);  
}  
  
console.log(processOperation(add)); // 15  
console.log(processOperation(multiply)); // 50
```

## 1.6 TypeScript Compiler

### Basic Usage

```
# Compile a TypeScript file  
tsc index.ts
```

```
# Compile with watch mode
tsc --watch index.ts

# Compile all TypeScript files
tsc

# Initialize tsconfig.json
tsc --init

# Compile with configuration
tsc --project tsconfig.json
```

Common tsc Options

| Option             | Description                                       |
|--------------------|---------------------------------------------------|
| --target           | Specify ECMAScript target (ES5, ES6, ES2020)      |
| --module           | Specify module code generation (commonjs, esnext) |
| --outDir           | Output directory for compiled JavaScript          |
| --rootDir          | Root directory of source files                    |
| --strict           | Enable all strict type-checking options           |
| --noImplicitAny    | Raise error on implicit 'any' types               |
| --strictNullChecks | Enable strict null checking                       |

Example tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true
  },
  "include": ["src/**/*.ts"],
}
```

```
"exclude": ["node_modules", "dist"]
}
```

## Day 1 Exercises

---

### Exercise 1.1: Type Annotations

**Instructions:** Add type annotations to the following variables.

```
// TODO: Add type annotations
let firstName = "John";
let age = 30;
let isActive = true;
let hobbies = ["reading", "gaming", "coding"];
let scores = [85, 90, 78];
let person = { name: "Alice", age: 25, city: "London" };
```

**Solution:**

```
// Add type annotations
let firstName: string = "John";
let age: number = 30;
let isActive: boolean = true;
let hobbies: string[] = ["reading", "gaming", "coding"];
let scores: number[] = [85, 90, 78];
let person: { name: string; age: number; city: string } = {
  name: "Alice",
  age: 25,
  city: "London"
};
```

### Exercise 1.2: Define Functions

**Instructions:** Write TypeScript functions with proper type annotations.

```
// 1. Function that doubles a number
function double(x: number): number {
  // Your code:
}

// 2. Function that greets a user
function greetUser(name: string, greeting?: string): string {
  // Your code:
}
```

```
}

// 3. Function that sums all numbers in an array
function sumArray(numbers: number[]): number {
  // Your code:
}

// 4. Function that creates a person object
function createPerson(name: string, age: number): { name: string; age: number } {
  // Your code:
}
```

### Solution:

```
function double(x: number): number {
  return x * 2;
}

function greetUser(name: string, greeting?: string): string {
  return greeting ? `${greeting}, ${name}!` : `Hello, ${name}!`;
}

function sumArray(numbers: number[]): number {
  return numbers.reduce((total, num) => total + num, 0);
}

function createPerson(name: string, age: number): { name: string; age: number } {
  return { name, age };
}
```

## Exercise 1.3: Tuples and Arrays

**Instructions:** Create the following data structures with proper types.

```
// 1. Tuple for a 2D coordinate
let coordinate: [number, number] = [];

// 2. Tuple for a person with name and age
let personInfo: [string, number] = [];

// 3. Array of strings for a todo list
let todoList: string[] = [];

// 4. Array of numbers for temperatures
let temperatures: number[] = [];
```

```
// 5. Tuple for RGB color
let rgbColor: [number, number, number] = [];
```

### Solution:

```
let coordinate: [number, number] = [10, 20];
let personInfo: [string, number] = ["Alice", 25];
let todoList: string[] = ["Learn TypeScript", "Build a project"];
let temperatures: number[] = [25.5, 30.2, 22.8];
let rgbColor: [number, number, number] = [255, 128, 0];
```

## Exercise 1.4: Type Inference

**Instructions:** Determine what types TypeScript infers for these variables.

```
let message = "Hello World";      // Type: _____
let count = 42;                   // Type: _____
let isReady = true;               // Type: _____
let items = [1, 2, 3, 4];         // Type: _____
let data = { id: 1, name: "test" }; // Type: _____
let result = 5 + "10";            // Type: _____
let answer = "yes" as const;      // Type: _____
```

### Solution:

```
let message = "Hello World";      // Type: string
let count = 42;                   // Type: number
let isReady = true;               // Type: boolean
let items = [1, 2, 3, 4];         // Type: number[]
let data = { id: 1, name: "test" }; // Type: { id: number; name: string; }
let result = 5 + "10";            // Type: string (number + string = string)
let answer = "yes" as const;      // Type: "yes" (literal type)
```

## Exercise 1.5: Function Types

**Instructions:** Write a function that accepts a callback and uses it.

```
// Write a function 'processNumbers' that takes an array of numbers
// and a callback function, and returns the processed array
function processNumbers(numbers: number[], callback: (num: number) => number): number[] {
  // Your code:
}
```

```
// Test with:  
// - Double each number  
// - Square each number  
// - Add 10 to each number
```

## Solution:

```
function processNumbers(numbers: number[], callback: (num: number) => number): number[]  
    return numbers.map(callback);  
}  
  
// Test  
const numbers = [1, 2, 3, 4, 5];  
  
const doubled = processNumbers(numbers, (n) => n * 2);  
const squared = processNumbers(numbers, (n) => n * n);  
const added = processNumbers(numbers, (n) => n + 10);  
  
console.log(doubled); // [2, 4, 6, 8, 10]  
console.log(squared); // [1, 4, 9, 16, 25]  
console.log(added);   // [11, 12, 13, 14, 15]
```

## Exercise 1.6: Create a TypeScript Project

**Instructions:** Set up a basic TypeScript project.

```
# 1. Create a new directory  
mkdir typescript-project  
cd typescript-project  
  
# 2. Initialize npm  
npm init -y  
  
# 3. Install TypeScript  
npm install typescript --save-dev  
  
# 4. Initialize tsconfig.json  
npx tsc --init  
  
# 5. Create src/index.ts  
# Add code with types  
  
# 6. Compile and run  
npx tsc  
node dist/index.js
```

## Sample src/index.ts:

```
// src/index.ts
interface Person {
  name: string;
  age: number;
}

function greet(person: Person): string {
  return `Hello, ${person.name}! You are ${person.age} years old.`;
}

const alice: Person = {
  name: "Alice",
  age: 25
};

console.log(greet(alice));
```

## Day 1 Summary

---

### Key Takeaways

1. **TypeScript:** JavaScript with static types. Catches errors at compile time.
2. **Basic Types:** `string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`
3. **Type Annotations:** `let name: string = "Alice";`
4. **Type Inference:** TypeScript can infer types automatically.
5. **Arrays:** `number[]` or `Array<number>`
6. **Tuples:** `[string, number]` - fixed-length, typed arrays
7. **Functions:** Typed parameters and return types: `function add(a: number, b: number): number`
8. **Compiler:** `tsc` compiles TypeScript to JavaScript.

### Quiz Yourself

1. What is TypeScript?
2. What are the basic types in TypeScript?
3. How do you annotate a variable with a type?
4. What is a tuple?
5. How do you define a function with typed parameters?
6. What does the `tsc` command do?

## Day 1 Homework

Create a TypeScript file that:

1. Defines a `Product` interface with `id`, `name`, `price`, and `inStock`
2. Creates an array of products
3. Defines a function to calculate the total price of all products
4. Defines a function to find products by name
5. Tests all functions with sample data

# Day 2: Advanced Types & Type Systems

---

## Learning Objectives

By the end of Day 2, you will be able to:

- Use union and intersection types
- Work with literal types
- Use type aliases and interfaces
- Use type assertions and type guards
- Understand the `any`, `unknown`, and `never` types

## 2.1 Union and Intersection Types

---

### Union Types ( `|` )

Union types allow a value to be one of several types.

```
// Union type: string or number
let value: string | number;
value = "Hello";    // Valid
value = 42;         // Valid
// value = true;    // Error: Type 'boolean' is not assignable to type 'string | number'

// Union type in functions
function formatValue(value: string | number): string {
    return `Value: ${value}`;
}

console.log(formatValue("Hello")); // "Value: Hello"
console.log(formatValue(42));      // "Value: 42"
```

```
// Union type with arrays
```

## Type Narrowing with Union Types

```
function process(value: string | number): void {  
  // Type narrowing using typeof  
  if (typeof value === "string") {  
    // Here, TypeScript knows value is a string  
    console.log(value.toUpperCase());  
  } else {  
    // Here, TypeScript knows value is a number  
    console.log(value.toFixed(2));  
  }  
}  
  
process("hello"); // "HELLO"  
process(42);      // "42.00"
```

## Intersection Types ( & )

Intersection types combine multiple types into one.

```
interface Person {  
  name: string;  
  age: number;  
}  
  
interface Employee {  
  employeeId: number;  
  department: string;  
}  
  
// Intersection: Person AND Employee  
type EmployeePerson = Person & Employee;  
  
const alice: EmployeePerson = {  
  name: "Alice",  
  age: 30,  
  employeeId: 12345,  
  department: "Engineering"  
};
```

## 2.2 Literal Types

---

## String Literal Types

```
// String literal type: only specific strings allowed
type Direction = "up" | "down" | "left" | "right";
type Status = "pending" | "success" | "error";
type Color = "red" | "green" | "blue";

let direction: Direction = "up"; // Valid
// let invalid: Direction = "forward"; // Error: Type '"forward"' is not assignable to

function setStatus(status: Status): void {
  console.log(`Status: ${status}`);
}

setStatus("success"); // Valid
// setStatus("failed"); // Error: Argument of type '"failed"' is not assignable to par
```

## Numeric Literal Types

```
type DiceValue = 1 | 2 | 3 | 4 | 5 | 6;
type HttpStatus = 200 | 404 | 500;

let roll: DiceValue = 4; // Valid
// let invalid: DiceValue = 7; // Error

let statusCode: HttpStatus = 404; // Valid
// statusCode = 301; // Error
```

## Boolean Literal Types

```
type TrueOnly = true;
type FalseOnly = false;

let yes: TrueOnly = true; // Valid
// let no: TrueOnly = false; // Error

type CompletionStatus = true | false; // Same as boolean
```

## Const Assertions

```
// Without const assertion - inferred as string
let greeting = "hello"; // Type: string

// With const assertion - inferred as literal
let literalGreeting = "hello" as const; // Type: "hello"
```

```
// Object with const assertion
const config = {
  env: "production",
  port: 3000
} as const;

// const config: { env: "production"; port: 3000; } = { env: "production", port: 3000 }
```

## 2.3 Type Aliases and Interfaces

### Type Aliases

Type aliases give a name to a type.

```
// Type alias for a primitive
type ID = string | number;

// Type alias for an object
type Person = {
  name: string;
  age: number;
  email?: string; // Optional property
};

// Type alias for a function
type GreetFunction = (name: string) => string;

// Using type aliases
let userId: ID = "abc123";
userId = 456; // Valid

let person: Person = {
  name: "Alice",
  age: 25,
  email: "alice@example.com" // Optional property
};

let greet: GreetFunction = (name) => `Hello, ${name}!`;
```

### Type Alias vs Interface

| Feature     | Type Alias                      | Interface                          |  |
|-------------|---------------------------------|------------------------------------|--|
| Declaration | type Person = { name: string; } | interface Person { name: string; } |  |

| Feature             | Type Alias                                                            | Interface                                   |                        |
|---------------------|-----------------------------------------------------------------------|---------------------------------------------|------------------------|
| Extends             | Intersection: <code>type Child = Person &amp; { age: number; }</code> | <code>interface Child extends Person</code> |                        |
| Implementation      | Union types: <code>type Value = string</code>                         | <code>number;</code>                        | Cannot represent union |
| Declaration merging | No                                                                    | Yes                                         |                        |
| Primary use         | Union types, complex types, primitives                                | Object shapes, class implementation         |                        |

## Interfaces

Interfaces define the shape of an object.

```
// Basic interface
interface Person {
  name: string;
  age: number;
  email?: string; // Optional property
}

// Interface with readonly property
interface Product {
  readonly id: number; // Cannot be changed after initialization
  name: string;
  price: number;
}

// Interface extending another interface
interface Student extends Person {
  studentId: string;
  courses: string[];
}

// Interface for function
interface Calculator {
  (a: number, b: number): number;
}

// Using interfaces
const person: Person = {
  name: "Alice",
  age: 25,
  email: "alice@example.com"
```

```
};

const student: Student = {
  name: "Bob",
  age: 20,
  studentId: "S12345",
  courses: ["Math", "Science"]
};

let add: Calculator = (a, b) => a + b;
```

## Index Signatures

```
// Interface with index signature
interface StringDictionary {
  [key: string]: string; // Any string key maps to a string value
}

const translations: StringDictionary = {
  hello: "hola",
  goodbye: "adiós"
};

// Interface with multiple index signatures
interface MixedDictionary {
  [key: string]: string | number;
  id: number; // Must match the index signature type
}

// Type alias with index signature
type NumberDictionary = {
  [key: string]: number;
};

const scores: NumberDictionary = {
  math: 95,
  science: 88,
  history: 76
};
```

## 2.4 Type Assertions

---

### What is a Type Assertion?

Type assertions tell TypeScript to treat a value as a specific type.

```
// Using 'as' syntax (preferred)
let someValue: unknown = "Hello, TypeScript!";
let strLength: number = (someValue as string).length;

// Using angle bracket syntax (not recommended with JSX)
let strLength2: number = (<string>someValue).length;

// Asserting a union type
function getLength(value: string | number): number {
  // Assert that value is a string
  return (value as string).length;
  // Or use type narrowing for safety
}
```

## When to Use Type Assertions

```
// 1. When working with DOM elements
const inputElement = document.getElementById("user-input") as HTMLInputElement;
inputElement.value = "New value";

// 2. When a third-party library has imprecise types
const response = await fetch("/api/data");
const data = await response.json() as { id: number; name: string };

// 3. When you know more about the type than TypeScript
const parsedData = JSON.parse('{ "id": 1, "name": "Alice" }') as Person;
```

## Double Assertion

```
// Converting one type to another via intermediate type
let value = "Hello" as unknown as number; // Dangerous! Avoid if possible
```

## 2.5 Type Guards

---

### What are Type Guards?

Type guards narrow the type of a variable within a conditional block.

#### **typeof** Type Guard

```
function processValue(value: string | number | boolean): string {
  if (typeof value === "string") {
    // TypeScript knows value is string
  }
}
```

```
        return value.toUpperCase();
    }

    if (typeof value === "number") {
        // TypeScript knows value is number
        return value.toFixed(2);
    }

    // TypeScript knows value is boolean
    return value ? "true" : "false";
}
```

## instanceof Type Guard

```
class Dog {
    bark(): void { console.log("Woof!"); }
}

class Cat {
    meow(): void { console.log("Meow!"); }
}

function handlePet(pet: Dog | Cat): void {
    if (pet instanceof Dog) {
        pet.bark(); // TypeScript knows pet is Dog
    } else {
        pet.meow(); // TypeScript knows pet is Cat
    }
}
```

## User-Defined Type Guards

```
// Type predicate: parameterName is Type
function isString(value: unknown): value is string {
    return typeof value === "string";
}

function isNumber(value: unknown): value is number {
    return typeof value === "number";
}

function processValue(value: unknown): void {
    if (isString(value)) {
        console.log(value.toUpperCase());
    } else if (isNumber(value)) {
        console.log(value.toFixed(2));
    } else {
        console.log("Unknown type");
    }
}
```

```
    }  
  }  
}
```

## in Operator Type Guard

```
interface Car {  
  drive(): void;  
}  
  
interface Boat {  
  sail(): void;  
}  
  
interface Amphibian extends Car, Boat {}  
  
function move(vehicle: Car | Boat): void {  
  if ("drive" in vehicle) {  
    vehicle.drive(); // TypeScript knows vehicle has drive method  
  } else {  
    vehicle.sail();  // TypeScript knows vehicle has sail method  
  }  
}
```

## 2.6 The any , unknown , and never Types

---

### any - The Wildcard

`any` disables type checking entirely. Use sparingly.

```
let anything: any = "Hello";  
anything = 42;  
anything = true;  
anything = { hello: "world" };  
  
// No type checking at all  
anything.someMethod(); // No error at compile time (but runtime error!)
```

### unknown - The Safe Alternative

`unknown` is like `any` but forces type checking before use.

```
let unknownValue: unknown = "Hello";  
  
// Error: Object is of type 'unknown'
```

```
// console.log(unknownValue.length);

// Must use type guards or assertions
if (typeof unknownValue === "string") {
    console.log(unknownValue.length); // Valid
}

// Or use type assertion
const strLength = (unknownValue as string).length;
```

## never - Unreachable Code

`never` represents values that never occur.

```
// Function never returns
function throwError(message: string): never {
    throw new Error(message);
}

function infiniteLoop(): never {
    while (true) {
        // never returns
    }
}

// Exhaustive checking with never
type Color = "red" | "green" | "blue";

function getColorHex(color: Color): string {
    switch (color) {
        case "red":
            return "#FF0000";
        case "green":
            return "#00FF00";
        case "blue":
            return "#0000FF";
        default:
            // TypeScript will error if not all cases are handled
            const exhaustiveCheck: never = color;
            return exhaustiveCheck;
    }
}
```

## Day 2 Exercises

---

### Exercise 2.1: Union Types

**Instructions:** Create a function that handles multiple types.

```
// 1. Define a union type for ID
type ID = string | number;

// 2. Create a function that accepts ID and returns a message
function getMessage(id: ID): string {
    // Your code:
}

// 3. Create a function that processes different status types
type Status = "loading" | "success" | "error" | "idle";

function getStatusMessage(status: Status): string {
    // Your code:
}
```

**Solution:**

```
type ID = string | number;

function getMessage(id: ID): string {
    if (typeof id === "string") {
        return `User ID: ${id}`;
    } else {
        return `User ID: #${id}`;
    }
}

type Status = "loading" | "success" | "error" | "idle";

function getStatusMessage(status: Status): string {
    switch (status) {
        case "loading": return "Loading...";
        case "success": return "Operation successful!";
        case "error": return "An error occurred!";
        case "idle": return "Ready";
    }
}
```

## Exercise 2.2: Interfaces and Type Aliases

**Instructions:** Define interfaces and type aliases for the following.

```
// 1. Define a User interface with name, age, and optional email
interface User {
    // Your code:
}
```

```
}

// 2. Define an Admin interface that extends User with role and permissions
interface Admin extends User {
  // Your code:
}

// 3. Define a type alias for a function that doubles a number
type DoubleFunction = (x: number) => number;

// 4. Create a product type with readonly id
type Product = {
  // Your code:
};
```

### Solution:

```
interface User {
  name: string;
  age: number;
  email?: string;
}

interface Admin extends User {
  role: "admin" | "super-admin";
  permissions: string[];
}

type DoubleFunction = (x: number) => number;

type Product = {
  readonly id: number;
  name: string;
  price: number;
  category: string;
  inStock: boolean;
};
```

## Exercise 2.3: Type Guards

**Instructions:** Implement type guards and use them.

```
// 1. Create a type guard for a Dog interface
interface Dog {
  breed: string;
  bark(): void;
}
```

```
function isDog(value: unknown): value is Dog {
    // Your code:
}

// 2. Create a type guard for a Cat interface
interface Cat {
    breed: string;
    meow(): void;
}

function isCat(value: unknown): value is Cat {
    // Your code:
}

// 3. Use type guards in a function
function handlePet(pet: unknown): void {
    // Your code:
}
```

### Solution:

```
interface Dog {
    breed: string;
    bark(): void;
}

function isDog(value: unknown): value is Dog {
    return typeof value === "object" &&
        value !== null &&
        "bark" in value &&
        typeof (value as Dog).bark === "function";
}

interface Cat {
    breed: string;
    meow(): void;
}

function isCat(value: unknown): value is Cat {
    return typeof value === "object" &&
        value !== null &&
        "meow" in value &&
        typeof (value as Cat).meow === "function";
}

function handlePet(pet: unknown): void {
    if (isDog(pet)) {
        console.log(`Dog of breed: ${pet.breed}`);
        pet.bark();
    } else if (isCat(pet)) {
        console.log(`Cat of breed: ${pet.breed}`);
    }
}
```

```

        pet.meow();
    } else {
        console.log("Unknown pet type");
    }
}

// Test
const dog: Dog = {
    breed: "Labrador",
    bark() { console.log("Woof!"); }
};

const cat: Cat = {
    breed: "Persian",
    meow() { console.log("Meow!"); }
};

handlePet(dog);
handlePet(cat);

```

## Exercise 2.4: any , unknown , and never

**Instructions:** Rewrite the following code using better types.

```

// 1. Replace 'any' with 'unknown'
function parseJsonString(value: any): void {
    // Your code:
}

// 2. Use 'never' for exhaustive checking
type PizzaSize = "small" | "medium" | "large";

function getPizzaPrice(size: PizzaSize): number {
    // Your code:
    // Throw error if size doesn't match any case
}

```

**Solution:**

```

function parseJsonString(value: unknown): void {
    if (typeof value === "string") {
        try {
            const parsed = JSON.parse(value);
            console.log("Parsed:", parsed);
        } catch (error) {
            console.log("Invalid JSON string");
        }
    } else {

```

```

        console.log("Not a string");
    }
}

type PizzaSize = "small" | "medium" | "large";

function getPizzaPrice(size: PizzaSize): number {
    switch (size) {
        case "small": return 10;
        case "medium": return 15;
        case "large": return 20;
        default:
            // Exhaustive check – ensures all cases are handled
            const exhaustiveCheck: never = size;
            return exhaustiveCheck;
    }
}

```

## Exercise 2.5: Complex Types

**Instructions:** Create complex type definitions.

```

// 1. Define a type for a blog post with:
//   - id (number)
//   - title (string)
//   - content (string)
//   - tags (array of strings, optional)
//   - published (boolean)
type BlogPost = {
    // Your code:
};

// 2. Define a type for a response object that can be:
//   - success: { data: T }
//   - error: { error: string }
//   - loading: { loading: true }
type Response<T> =
    // Your code:
;

// 3. Define a type for a configuration object with:
//   - env: "development" | "staging" | "production"
//   - apiUrl: string
//   - timeout: number
type Config = {
    // Your code:
};

```

**Solution:**

```
type BlogPost = {
  id: number;
  title: string;
  content: string;
  tags?: string[];
  published: boolean;
};

type Response<T> =
  | { success: true; data: T }
  | { success: false; error: string }
  | { loading: true };

type Config = {
  env: "development" | "staging" | "production";
  apiUrl: string;
  timeout: number;
};
```

## Day 2 Summary

---

### Key Takeaways

1. **Union Types ( | )**: Value can be one of several types
2. **Intersection Types ( & )**: Combine multiple types
3. **Literal Types**: Specific values as types ( "success" | "error" )
4. **Type Aliases**: type Name = Type;
5. **Interfaces**: Define object shapes with optional and readonly properties
6. **Type Guards**: Narrow types using typeof , instanceof , custom guards
7. **Type Assertions**: as Type - tell TypeScript what you know
8. **any** : Disable type checking (avoid)
9. **unknown** : Safe alternative to any
10. **never** : Values that never occur

### Quiz Yourself

1. What is the difference between union and intersection types?
2. What is a literal type?
3. When would you use a type alias vs an interface?
4. What is a type guard?
5. What is the difference between any and unknown ?
6. When would you use the never type?

## Day 2 Homework

Create a TypeScript file that:

1. Defines a type for a `Shape` (circle or rectangle)
2. Defines a `Circle` type with `radius` property
3. Defines a `Rectangle` type with `width` and `height`
4. Defines a function that calculates the area of a shape
5. Uses type guards to handle different shapes

# Day 3: Classes, Interfaces & Object-Oriented Programming

---

## Learning Objectives

By the end of Day 3, you will be able to:

- Create classes with TypeScript
- Use access modifiers (public, private, protected)
- Implement interfaces in classes
- Use abstract classes
- Understand inheritance and polymorphism

## 3.1 Classes in TypeScript

---

### Basic Class

```
class Person {  
  // Properties  
  name: string;  
  age: number;  
  
  // Constructor  
  constructor(name: string, age: number) {  
    this.name = name;  
    this.age = age;  
  }  
  
  // Method  
  greet(): string {  
    return `Hello, I'm ${this.name} and I'm ${this.age} years old.`;  
  }  
}
```

```
    }  
  }  
  
  // Create instance  
  const alice = new Person("Alice", 25);  
  console.log(alice.greet()); // "Hello, I'm Alice and I'm 25 years old."
```

## Simplified Constructor with Parameter Properties

```
class Person {  
  // Parameters automatically become class properties  
  constructor(  
    public name: string,  
    public age: number,  
    private id: string,  
    protected role: string  
  ) {  
    // No need to assign manually  
  }  
  
  getInfo(): string {  
    return `${this.name}, ${this.age}, ${this.role}`;  
  }  
}
```

## Access Modifiers

| Modifier  | Description                                |
|-----------|--------------------------------------------|
| public    | Accessible anywhere (default)              |
| private   | Accessible only within the class           |
| protected | Accessible within the class and subclasses |
| readonly  | Can only be set at initialization          |

```
class BankAccount {  
  // Readonly property  
  public readonly accountNumber: string;  
  
  // Private property  
  private balance: number;  
  
  // Protected property  
  protected owner: string;  
  
  // Public property  
  public type: string;
```

```

    constructor(accountNumber: string, initialBalance: number, owner: string, type: st
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.owner = owner;
        this.type = type;
    }

    // Public method
    deposit(amount: number): void {
        if (amount <= 0) {
            throw new Error("Amount must be positive");
        }
        this.balance += amount;
    }

    // Public method with private helper
    withdraw(amount: number): boolean {
        if (this.canWithdraw(amount)) {
            this.balance -= amount;
            return true;
        }
        return false;
    }

    // Private method
    private canWithdraw(amount: number): boolean {
        return amount > 0 && amount <= this.balance;
    }

    // Public getter
    getBalance(): number {
        return this.balance;
    }

    // Protected method (accessible in subclasses)
    protected logTransaction(amount: number, type: string): void {
        console.log(`Transaction: ${type} of ${amount} on account ${this.accountNumber

```

## Readonly Properties

```

class Configuration {
    public readonly version: string = "1.0.0";
    public readonly apiUrl: string;

    constructor(apiUrl: string) {
        this.apiUrl = apiUrl;
    }
}

```

```
const config = new Configuration("https://api.example.com");
console.log(config.version); // "1.0.0"
// ...
```

## 3.2 Getters and Setters

---

```
class Temperature {
    private _celsius: number = 0;

    // Getter
    get celsius(): number {
        return this._celsius;
    }

    // Setter with validation
    set celsius(value: number) {
        if (value < -273.15) {
            throw new Error("Temperature below absolute zero");
        }
        this._celsius = value;
    }

    // Computed property
    get fahrenheit(): number {
        return (this._celsius * 9/5) + 32;
    }

    set fahrenheit(value: number) {
        this.celsius = (value - 32) * 5/9;
    }
}

const temp = new Temperature();
temp.celsius = 25;
console.log(temp.celsius); // 25
console.log(temp.fahrenheit); // 77

temp.fahrenheit = 68;
console.log(temp.celsius); // 20
```

## 3.3 Interfaces with Classes

---

### Class Implementing Interface

```
// Interface definition
interface User {
  id: number;
  name: string;
  email: string;
  login(): void;
  logout(): void;
}

// Class implementing interface
class Customer implements User {
  id: number;
  name: string;
  email: string;

  constructor(id: number, name: string, email: string) {
    this.id = id;
    this.name = name;
    this.email = email;
  }

  login(): void {
    console.log(`${this.name} logged in`);
  }

  logout(): void {
    console.log(`${this.name} logged out`);
  }
}

const customer = new Customer(1, "Alice", "alice@example.com");
customer.login();
customer.logout();
```

## Multiple Interfaces

```
interface Person {
  name: string;
  age: number;
}

interface Worker {
  employeeId: string;
  department: string;
}

// Class implementing multiple interfaces
class Employee implements Person, Worker {
  name: string;
  age: number;
```

```
employeeId: string;
department: string;

constructor(name: string, age: number, employeeId: string, department: string) {
    this.name = name;
    this.age = age;
    this.employeeId = employeeId;
    this.department = department;
}

getInfo(): string {
    return `${this.name}, ${this.age}, ${this.employeeId}, ${this.department}`;
}
```

## Interface Extending Interface

```
interface Animal {
    name: string;
    makeSound(): void;
}

interface Dog extends Animal {
    breed: string;
    fetch(): void;
}

class GoldenRetriever implements Dog {
    name: string;
    breed: string;

    constructor(name: string) {
        this.name = name;
        this.breed = "Golden Retriever";
    }

    makeSound(): void {
        console.log("Woof!");
    }

    fetch(): void {
        console.log(`${this.name} is fetching the ball!`);
    }
}
```

## 3.4 Inheritance

---

## Base Class and Derived Class

```
// Base class
class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    makeSound(): void {
        console.log("Some animal sound");
    }

    getName(): string {
        return this.name;
    }
}

// Derived class
class Dog extends Animal {
    private breed: string;

    constructor(name: string, breed: string) {
        super(name); // Call parent constructor
        this.breed = breed;
    }

    // Override parent method
    makeSound(): void {
        console.log(`${this.name} says: Woof!`);
    }

    // New method
    fetch(): void {
        console.log(`${this.name} is fetching`);
    }
}

const dog = new Dog("Rex", "Labrador");
dog.makeSound(); // "Rex says: Woof!"
dog.fetch();     // "Rex is fetching"
console.log(dog.getName()); // "Rex"
```

## Method Override

```
class Shape {
    protected color: string;

    constructor(color: string) {
```

```
        this.color = color;
    }

    // To be overridden
    getArea(): number {
        return 0;
    }

    getDescription(): string {
        return `A ${this.color} shape`;
    }
}

class Circle extends Shape {
    private radius: number;

    constructor(color: string, radius: number) {
        super(color);
        this.radius = radius;
    }

    // Override getArea
    getArea(): number {
        return Math.PI * this.radius * this.radius;
    }

    // Override getDescription
    getDescription(): string {
        return `A ${this.color} circle with radius ${this.radius}`;
    }
}

class Rectangle extends Shape {
    private width: number;
    private height: number;

    constructor(color: string, width: number, height: number) {
        super(color);
        this.width = width;
        this.height = height;
    }

    getArea(): number {
        return this.width * this.height;
    }

    getDescription(): string {
        return `A ${this.color} rectangle (${this.width}x${this.height})`;
    }
}

const circle = new Circle("red", 5);
console.log(circle.getArea());           // 78.53981633974483
```

```
console.log(circle.getDescription()); // "A red circle with radius 5"

const rectangle = new Rectangle("blue", 4, 6);
console.log(rectangle.getArea());      // 24
console.log(rectangle.getDescription()); // "A blue rectangle (4x6)"
```

## super Keyword

```
class Vehicle {
  protected make: string;
  protected model: string;

  constructor(make: string, model: string) {
    this.make = make;
    this.model = model;
  }

  getDetails(): string {
    return `${this.make} ${this.model}`;
  }
}

class Car extends Vehicle {
  private doors: number;

  constructor(make: string, model: string, doors: number) {
    super(make, model);
    this.doors = doors;
  }

  // Use super to call parent method
  getDetails(): string {
    return `${super.getDetails()} with ${this.doors} doors`;
  }
}

const car = new Car("Toyota", "Camry", 4);
console.log(car.getDetails()); // "Toyota Camry with 4 doors"
```

## 3.5 Abstract Classes

---

### What is an Abstract Class?

Abstract classes cannot be instantiated directly. They define a template for other classes.

```
// Abstract class
abstract class Shape {
    protected color: string;

    constructor(color: string) {
        this.color = color;
    }

    // Abstract method (must be implemented by subclasses)
    abstract getArea(): number;

    // Concrete method (can be used as-is or overridden)
    getDescription(): string {
        return `A ${this.color} shape`;
    }
}

// Subclass implementing abstract method
class Square extends Shape {
    private side: number;

    constructor(color: string, side: number) {
        super(color);
        this.side = side;
    }

    // Must implement getArea
    getArea(): number {
        return this.side * this.side;
    }
}

class Triangle extends Shape {
    private base: number;
    private height: number;

    constructor(color: string, base: number, height: number) {
        super(color);
        this.base = base;
        this.height = height;
    }

    getArea(): number {
        return (this.base * this.height) / 2;
    }
}

// Cannot instantiate abstract class
// const shape = new Shape("red"); // Error: Cannot create an instance of an abstract

const square = new Square("red", 5);
console.log(square.getArea()); // 25
```

```
console.log(square.getDescription()); // "A red shape"
```

```
const triangle = new Triangle("blue", 4, 3);
```

## Abstract Methods and Properties

```
abstract class Database {
  // Abstract property
  abstract connectionString: string;

  // Abstract method
  abstract connect(): void;
  abstract disconnect(): void;

  // Concrete method
  query(sql: string): void {
    console.log(`Executing query: ${sql}`);
  }
}

class PostgresDB extends Database {
  connectionString: string;

  constructor(connectionString: string) {
    super();
    this.connectionString = connectionString;
  }

  connect(): void {
    console.log(`Connecting to Postgres at ${this.connectionString}`);
  }

  disconnect(): void {
    console.log("Disconnecting from Postgres");
  }
}

const db = new PostgresDB("postgres://localhost:5432/mydb");
db.connect();
db.query("SELECT * FROM users");
db.disconnect();
```

## 3.6 Polymorphism

```
// Base class
abstract class Shape {
```

```
    abstract getArea(): number;
    abstract getDescription(): string;
}

// Subclasses
class Circle extends Shape {
    constructor(private radius: number) {
        super();
    }

    getArea(): number {
        return Math.PI * this.radius * this.radius;
    }

    getDescription(): string {
        return `Circle with radius ${this.radius}`;
    }
}

class Rectangle extends Shape {
    constructor(private width: number, private height: number) {
        super();
    }

    getArea(): number {
        return this.width * this.height;
    }

    getDescription(): string {
        return `Rectangle (${this.width}x${this.height})`;
    }
}

// Polymorphic function
function printShapeInfo(shape: Shape): void {
    console.log(`Shape: ${shape.getDescription()}`);
    console.log(`Area: ${shape.getArea().toFixed(2)}`);
}

// Usage
const shapes: Shape[] = [
    new Circle(5),
    new Rectangle(4, 6),
    new Circle(3)
];

shapes.forEach(shape => {
    printShapeInfo(shape);
});

// Output:
// Shape: Circle with radius 5
// Area: 78.54
```

```
// Shape: Rectangle (4x6)
// Area: 24.00
// Shape: Circle with radius 3
// Area: 28.27
```

## Day 3 Exercises

---

### Exercise 3.1: Create a Class

**Instructions:** Create a `Book` class with properties and methods.

```
class Book {
  // Properties: title (string), author (string), pages (number), isRead (boolean)
  // Your code:

  // Constructor
  // Your code:

  // Method: read() - marks book as read and returns a message
  // Your code:

  // Method: getInfo() - returns book information
  // Your code:
}
```

**Solution:**

```
class Book {
  private title: string;
  private author: string;
  private pages: number;
  private isRead: boolean;

  constructor(title: string, author: string, pages: number) {
    this.title = title;
    this.author = author;
    this.pages = pages;
    this.isRead = false;
  }

  read(): string {
    this.isRead = true;
    return `You have read "${this.title}" by ${this.author}`;
  }
}
```

```

    getInfo(): string {
        const status = this.isRead ? "Read" : "Not read yet";
        return `${this.title} by ${this.author} - ${this.pages} pages - ${status}`;
    }
}

const book = new Book("1984", "George Orwell", 328);
console.log(book.getInfo()); // "1984" by George Orwell - 328 pages - Not read yet
console.log(book.read());    // "You have read "1984" by George Orwell"
// "1984" by George Orwell - 328 pages - Read

```

## Exercise 3.2: Inheritance

**Instructions:** Create a base class and derived classes.

```

// 1. Create a Vehicle base class with make, model, year
class Vehicle {
    // Your code:
}

// 2. Create a Car class that extends Vehicle with doors
class Car extends Vehicle {
    // Your code:
}

// 3. Create a Motorcycle class that extends Vehicle with type
class Motorcycle extends Vehicle {
    // Your code:
}

```

**Solution:**

```

class Vehicle {
    protected make: string;
    protected model: string;
    protected year: number;

    constructor(make: string, model: string, year: number) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    getInfo(): string {
        return `${this.year} ${this.make} ${this.model}`;
    }
}

```

```
class Car extends Vehicle {
  private doors: number;

  constructor(make: string, model: string, year: number, doors: number) {
    super(make, model, year);
    this.doors = doors;
  }

  getInfo(): string {
    return `${super.getInfo()} - ${this.doors} doors`;
  }
}

class Motorcycle extends Vehicle {
  private type: string;

  constructor(make: string, model: string, year: number, type: string) {
    super(make, model, year);
    this.type = type;
  }

  getInfo(): string {
    return `${super.getInfo()} - ${this.type}`;
  }
}

const car = new Car("Toyota", "Camry", 2022, 4);
const bike = new Motorcycle("Harley", "Davidson", 2021, "Cruiser");

console.log(car.getInfo()); // "2022 Toyota Camry - 4 doors"
console.log(bike.getInfo()); // "2021 Harley Davidson - Cruiser"
```

## Exercise 3.3: Abstract Class

**Instructions:** Create an abstract class and subclasses.

```
// 1. Create an abstract Employee class with name, salary
abstract class Employee {
  // Your code:
}

// 2. Create a Manager class that extends Employee
class Manager extends Employee {
  // Your code:
}

// 3. Create a Developer class that extends Employee
class Developer extends Employee {
```

```
// Your code:  
}
```

## Solution:

```
abstract class Employee {  
    protected name: string;  
    protected salary: number;  
  
    constructor(name: string, salary: number) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    abstract getRole(): string;  
    abstract getBonus(): number;  
  
    getInfo(): string {  
        return `${this.getRole(): ${this.name} - ${this.salary}`;  
    }  
}  
  
class Manager extends Employee {  
    private teamSize: number;  
  
    constructor(name: string, salary: number, teamSize: number) {  
        super(name, salary);  
        this.teamSize = teamSize;  
    }  
  
    getRole(): string {  
        return "Manager";  
    }  
  
    getBonus(): number {  
        return this.salary * 0.2;  
    }  
  
    getInfo(): string {  
        return `${super.getInfo()} - Managing ${this.teamSize} people`;  
    }  
}  
  
class Developer extends Employee {  
    private language: string;  
  
    constructor(name: string, salary: number, language: string) {  
        super(name, salary);  
        this.language = language;  
    }  
}
```

```

    getRole(): string {
        return "Developer";
    }

    getBonus(): number {
        return this.salary * 0.1;
    }

    getInfo(): string {
        return `${super.getInfo()} - ${this.language} developer`;
    }
}

const manager = new Manager("Alice", 80000, 5);
const developer = new Developer("Bob", 60000, "TypeScript");

console.log(manager.getInfo()); // "Manager: Alice - $80000 - Managing 5 people"
console.log(developer.getInfo()); // "Developer: Bob - $60000 - TypeScript developer"
console.log(`Bonus: ${manager.getBonus()}`); // "Bonus: $16000"

```

## Exercise 3.4: Interface Implementation

**Instructions:** Implement interfaces in classes.

```

// 1. Define a Printable interface with print() method
interface Printable {
    // Your code:
}

// 2. Define a Scannable interface with scan() method
interface Scannable {
    // Your code:
}

// 3. Create a Document class that implements both
class Document implements Printable, Scannable {
    // Your code:
}

```

**Solution:**

```

interface Printable {
    print(): void;
}

interface Scannable {
    scan(): void;
}

```

```
}

class Document implements Printable, Scannable {
    private content: string;

    constructor(content: string) {
        this.content = content;
    }

    print(): void {
        console.log(`Printing: ${this.content}`);
    }

    scan(): void {
        console.log(`Scanning: ${this.content}`);
    }
}

const doc = new Document("Hello World");
doc.print(); // "Printing: Hello World"
doc.scan();  // "Scanning: Hello World"
```

## Day 3 Summary

---

### Key Takeaways

1. **Classes:** Blueprint for objects with properties and methods
2. **Access Modifiers:** `public`, `private`, `protected`, `readonly`
3. **Getters/Setters:** Control access to properties
4. **Inheritance:** `extends` keyword, `super()` for parent constructor
5. **Interfaces:** Define contracts that classes implement
6. **Abstract Classes:** Cannot be instantiated, define templates
7. **Polymorphism:** Same interface, different implementations

### Quiz Yourself

1. What are the access modifiers in TypeScript?
2. What is the difference between `private` and `protected`?
3. How do you implement an interface in a class?
4. What is an abstract class?
5. What is polymorphism?

## Day 3 Homework

Create a TypeScript file that:

1. Defines an abstract `Animal` class with `name` and abstract `makeSound()` method
2. Creates `Dog`, `Cat`, and `Bird` classes that extend `Animal`
3. Creates an array of animals and calls `makeSound()` on each
4. Uses polymorphism to handle different animal types

# Day 4: Generics, Utility Types & Advanced Patterns

---

## Learning Objectives

By the end of Day 4, you will be able to:

- Use generics to create reusable components
- Work with generic functions and classes
- Use built-in utility types
- Create custom utility types
- Understand advanced TypeScript patterns

## 4.1 Generics

---

### What Are Generics?

Generics allow you to create reusable components that work with different types.

### Generic Functions

```
// Generic function with type parameter T
function identity<T>(value: T): T {
    return value;
}

// Usage
const num = identity<number>(42);    // Type: number
const str = identity<string>("Hello"); // Type: string

// Type inference (TypeScript can infer the type)
const inferred = identity(42);        // Type: number

// Generic function with multiple type parameters
```

```
function pair<T, U>(first: T, second: U): [T, U] {
    return [first, second];
}

const p = pair("Alice", 25); // Type: [string, number]
```

## Generic Type Constraints

```
// Constraint with extends
function getLength<T extends { length: number }>(value: T): number {
    return value.length;
}

console.log(getLength("Hello")); // 5
console.log(getLength([1, 2, 3])); // 3
console.log(getLength({ length: 10 })); // 10
// console.log(getLength(42)); // Error: Argument of type 'number' is not assignable to parameter of type '{ length: number; }'

// Constraint with keyof
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const person = { name: "Alice", age: 25 };
console.log(getProperty(person, "name")); // "Alice"
console.log(getProperty(person, "age")); // 25
// console.log(getProperty(person, "city")); // Error: Argument of type '"city"' is not assignable to parameter of type 'keyof { name: string; age: number; }'
```

## Generic Interfaces

```
// Generic interface
interface Box<T> {
    value: T;
    getValue(): T;
}

// Implementing generic interface
class NumberBox implements Box<number> {
    value: number;

    constructor(value: number) {
        this.value = value;
    }

    getValue(): number {
        return this.value;
    }
}
```

```
}

class StringBox implements Box<string> {
  value: string;

  constructor(value: string) {
    this.value = value;
  }

  getValue(): string {
    return this.value;
  }
}

const numBox = new NumberBox(42);
console.log(numBox.getValue()); // 42

const strBox = new StringBox("Hello");
console.log(strBox.getValue()); // "Hello"
```

## Generic Classes

```
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  peek(): T | undefined {
    return this.items[this.items.length - 1];
  }

  isEmpty(): boolean {
    return this.items.length === 0;
  }

  size(): number {
    return this.items.length;
  }
}

// Usage
const numberStack = new Stack<number>();
numberStack.push(1);
numberStack.push(2);
numberStack.push(3);
```

```
console.log(numberStack.pop()); // 3
console.log(numberStack.peek()); // 2
console.log(numberStack.size()); // 2

const stringStack = new Stack<string>();
stringStack.push("Hello");
stringStack.push("World");
console.log(stringStack.pop()); // "World"
```

## Generic Type Aliases

```
// Generic type alias
type Result<T> = {
  success: boolean;
  data?: T;
  error?: string;
};

function fetchData<T>(url: string): Promise<Result<T>> {
  return fetch(url)
    .then(res => res.json())
    .then(data => ({
      success: true,
      data
    }))
    .catch(error => ({
      success: false,
      error: error.message
    }));
}

// Usage
interface User {
  id: number;
  name: string;
}

const userResult = await fetchData<User>("/api/users/1");
if (userResult.success) {
  console.log(userResult.data?.name);
} else {
  console.log(userResult.error);
}
```

## 4.2 Utility Types

---

### What are Utility Types?

Built-in types for common type transformations.

## Partial

Makes all properties optional.

```
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
}

// Partial - all properties optional
type PartialUser = Partial<User>;
// Equivalent to: { id?: number; name?: string; email?: string; age?: number; }

const updateUser = (id: number, updates: Partial<User>): void => {
  console.log(`Updating user ${id} with:`, updates);
};

updateUser(1, { name: "Alice" }); // Valid - only name provided
updateUser(1, { email: "alice@example.com", age: 26 }); // Valid
```

## Required

Makes all properties required.

```
interface Config {
  url?: string;
  timeout?: number;
  retries?: number;
}

// Required - all properties required
type RequiredConfig = Required<Config>;
// Equivalent to: { url: string; timeout: number; retries: number; }

const config: RequiredConfig = {
  url: "https://api.example.com",
  timeout: 5000,
  retries: 3
};
```

## Readonly

Makes all properties readonly.

```
interface User {
  id: number;
  name: string;
}

// Readonly - all properties readonly
type ReadonlyUser = Readonly<User>;
// Equivalent to: { readonly id: number; readonly name: string; }

const user: ReadonlyUser = { id: 1, name: "Alice" };
// user.id = 2; // Error: Cannot assign to 'id' because it is a read-only property
// user.name = "Bob"; // Error: Cannot assign to 'name' because it is a read-only pro
```

## Record<K, T>

Creates an object type with keys of type K and values of type T.

```
type Status = "success" | "error" | "pending";
type StatusMessage = Record<Status, string>;

const messages: StatusMessage = {
  success: "Operation successful!",
  error: "Operation failed!",
  pending: "Operation in progress..."
};

// Record with number keys
type NumberRecord = Record<number, string>;
const colors: NumberRecord = {
  1: "red",
  2: "green",
  3: "blue"
};
```

## Pick<T, K>

Selects a subset of properties.

```
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
  address: string;
}
```

```
// Pick – select specific properties
type UserBasic = Pick<User, "id" | "name">;
// Equivalent to: { id: number; name: string; }

const user: UserBasic = {
  id: 1,
  name: "Alice"
  // email, age, address not required
};
```

## Omit<T, K>

Excludes a subset of properties.

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  age: number;
}

// Omit – exclude specific properties
type UserPublic = Omit<User, "password">;
// Equivalent to: { id: number; name: string; email: string; age: number; }

const publicUser: UserPublic = {
  id: 1,
  name: "Alice",
  email: "alice@example.com",
  age: 25
  // password not included
};
```

## Exclude<T, U>

Excludes types from T that are assignable to U.

```
type T = "a" | "b" | "c" | "d";
type U = "b" | "d";

type Result = Exclude<T, U>;
// Equivalent to: "a" | "c"

type NonString = Exclude<string | number | boolean, string>;
// Equivalent to: number | boolean
```

## Extract<T, U>

Extracts types from T that are assignable to U.

```
type T = "a" | "b" | "c" | "d";
type U = "b" | "d" | "e";

type Result = Extract<T, U>;
// Equivalent to: "b" | "d"

type StringType = Extract<string | number | boolean, string>;
// Equivalent to: string
```

## NonNullable

Excludes null and undefined.

```
type Nullable = string | number | null | undefined;
type NonNullableType = NonNullable<Nullable>;
// Equivalent to: string | number

const value: NonNullableType = "Hello";
// value = null; // Error: Type 'null' is not assignable to type 'string | number'
```

## ReturnType

Gets the return type of a function.

```
function createUser(name: string, age: number): { id: number; name: string; age: number } {
    return { id: 1, name, age };
}

type UserType = ReturnType<typeof createUser>;
// Equivalent to: { id: number; name: string; age: number; }

const user: UserType = {
    id: 1,
    name: "Alice",
    age: 25
};
```

## Parameters

Gets the parameter types of a function.

```
function createUser(name: string, age: number): { id: number; name: string; age: number } {
    return { id: 1, name, age };
}
```

```

    }

    type CreateUserParams = Parameters<typeof createUser>;
    // Equivalent to: [string, number]

    const params: CreateUserParams = ["Alice", 25];

```

## 4.3 Custom Utility Types

### Creating Your Own Utility Types

```

// Make all properties optional and nullable
type Nullable<T> = {
    [P in keyof T]: T[P] | null;
};

// Make all properties optional and readonly
type ReadonlyPartial<T> = {
    readonly [P in keyof T]?: T[P];
};

// Deep readonly (recursive)
type DeepReadonly<T> = {
    readonly [P in keyof T]: T[P] extends object ? DeepReadonly<T[P]> : T[P];
};

// Extract keys with value of a specific type
type KeysOfType<T, U> = {
    [P in keyof T]: T[P] extends U ? P : never;
}[keyof T];

// Usage
interface User {
    id: number;
    name: string;
    email: string;
    age: number;
    address: {
        street: string;
        city: string;
        zip: string;
    };
}

// Make all properties nullable
type NullableUser = Nullable<User>;

```

```
// Make all properties optional and readonly
type ReadonlyPartialUser = ReadonlyPartial<User>;

// Deep readonly
type DeepReadonlyUser = DeepReadonly<User>;

// Get string keys
type StringKeys = KeysOfType<User, string>;
// Equivalent to: "name" | "email" | "street" | "city" | "zip"
```

## Mapped Types

```
// Mapped type - transform all properties
type Transform<T> = {
  [P in keyof T]: T[P] | string;
};

// Mapped type with conditional types
type Stringify<T> = {
  [P in keyof T]: T[P] extends string ? string : number;
};

// Mapped type with modifiers
type Mutable<T> = {
  -readonly [P in keyof T]: T[P];
};

type Optional<T> = {
  [P in keyof T]?: T[P];
};

type RequiredProperties<T> = {
  [P in keyof T]-?: T[P];
};
```

## 4.4 Conditional Types

---

### Basic Conditional Types

```
// Conditional type: T extends U ? X : Y
type IsString<T> = T extends string ? true : false;

type Result1 = IsString<string>; // true
type Result2 = IsString<number>; // false
type Result3 = IsString<"hello">; // true
```

```
// Conditional type with never
type ExcludeString<T> = T extends string ? never : T;

type Result = ExcludeString<string | number | boolean>;
// Equivalent to: number | boolean
```

## Distributive Conditional Types

```
// Distributive conditionals work with union types
type ToArray<T> = T extends any ? T[] : never;

type Result = ToArray<string | number>;
// Equivalent to: string[] | number[]

// Better version
type ToArrayNonDistributive<T> = T[];

type Result2 = ToArrayNonDistributive<string | number>;
// Equivalent to: (string | number)[]
```

## 4.5 Type Inference with `infer`

---

```
// Extract return type using infer
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function greet(name: string): string {
    return `Hello, ${name}!`;
}

type GreetReturn = ReturnType<typeof greet>; // string

// Extract parameter types using infer
type Parameters<T> = T extends (...args: infer P) => any ? P : never;

type GreetParams = Parameters<typeof greet>; // [string]

// Extract element type from array
type ElementType<T> = T extends (infer U)[] ? U : never;

type ArrayElement = ElementType<string[]>; // string
type NumberElement = ElementType<number[]>; // number
```

## 4.6 Template Literal Types

---

```
// Template literal types
type EventName = "click" | "hover" | "focus";
type EventHandler = `on${Capitalize<EventName>}`;
// Equivalent to: "onClick" | "onHover" | "onFocus"

// String manipulation types
type UppercaseGreeting = Uppercase<"hello">; // "HELLO"
type LowercaseGreeting = Lowercase<"HELLO">; // "hello"
type CapitalizeGreeting = Capitalize<"hello">; // "Hello"
type UncapitalizeGreeting = Uncapitalize<"Hello">; // "hello"

// Practical example
type HTTPMethod = "get" | "post" | "put" | "delete";
type Endpoint = `/${string}`;

type Route = {
  [M in HTTPMethod as Uppercase<M>]: Endpoint;
};
// Equivalent to: { GET: Endpoint; POST: Endpoint; PUT: Endpoint; DELETE: Endpoint; }

// Combined with utilities
type ApiResponse<T> = {
  data: T;
  status: number;
};

type ApiEndpoint<Method extends HTTPMethod, Path extends string> = {
  method: Method;
  path: `/${Path}`;
  response: ApiResponse<any>;
};
```

## Day 4 Exercises

### Exercise 4.1: Generic Functions

**Instructions:** Write generic functions for the following tasks.

```
// 1. A generic function that returns the first element of an array
function firstElement<T>(array: T[]): T | undefined {
  // Your code:
}

// 2. A generic function that returns the last element of an array
function lastElement<T>(array: T[]): T | undefined {
  // Your code:
}
```

```
}

// 3. A generic function that reverses an array
function reverseArray<T>(array: T[]): T[] {
    // Your code:
}

// 4. A generic function that filters by a predicate
function filterArray<T>(array: T[], predicate: (item: T) => boolean): T[] {
    // Your code:
}
```

### Solution:

```
function firstElement<T>(array: T[]): T | undefined {
    return array[0];
}

function lastElement<T>(array: T[]): T | undefined {
    return array[array.length - 1];
}

function reverseArray<T>(array: T[]): T[] {
    return array.slice().reverse();
}

function filterArray<T>(array: T[], predicate: (item: T) => boolean): T[] {
    return array.filter(predicate);
}

// Test
const numbers = [1, 2, 3, 4, 5];
const names = ["Alice", "Bob", "Charlie"];

console.log(firstElement(numbers)); // 1
console.log(lastElement(numbers)); // 5
console.log(reverseArray(numbers)); // [5, 4, 3, 2, 1]
console.log(filterArray(numbers, (n) => n > 3)); // [4, 5]
console.log(filterArray(names, (n) => n.length > 3)); // ["Alice", "Charlie"]
```

## Exercise 4.2: Generic Classes

**Instructions:** Create generic classes for the following.

```
// 1. A generic Queue class (FIFO)
class Queue<T> {
    private items: T[] = [];
```

```
// enqueue: adds item to the end
enqueue(item: T): void {
    // Your code:
}

// dequeue: removes and returns the first item
dequeue(): T | undefined {
    // Your code:
}

// peek: returns the first item without removing
peek(): T | undefined {
    // Your code:
}

// size: returns the number of items
size(): number {
    // Your code:
}

// isEmpty: returns true if queue is empty
isEmpty(): boolean {
    // Your code:
}
}

// 2. A generic KeyValuePair class
class KeyValuePair<K, V> {
    // Your code:
}
```

## Solution:

```
class Queue<T> {
    private items: T[] = [];

    enqueue(item: T): void {
        this.items.push(item);
    }

    dequeue(): T | undefined {
        return this.items.shift();
    }

    peek(): T | undefined {
        return this.items[0];
    }

    size(): number {
        return this.items.length;
    }
}
```

```

    isEmpty(): boolean {
        return this.items.length === 0;
    }
}

class KeyValuePair<K, V> {
    private key: K;
    private value: V;

    constructor(key: K, value: V) {
        this.key = key;
        this.value = value;
    }

    getKey(): K {
        return this.key;
    }

    getValue(): V {
        return this.value;
    }

    setValue(value: V): void {
        this.value = value;
    }
}

// Test
const queue = new Queue<string>();
queue.enqueue("First");
queue.enqueue("Second");
queue.enqueue("Third");
console.log(queue.peek());    // "First"
console.log(queue.dequeue()); // "First"
console.log(queue.size());    // 2

const pair = new KeyValuePair("name", "Alice");
console.log(pair.getKey());    // "name"
console.log(pair.getValue());  // "Alice"
pair.setValue("Bob");
console.log(pair.getValue());  // "Bob"

```

## Exercise 4.3: Utility Types

**Instructions:** Use TypeScript utility types to solve these problems.

```

// 1. Create a type that makes all properties of User optional
interface User {

```

```
    id: number;
    name: string;
    email: string;
    age: number;
}
type OptionalUser = // Your code:

// 2. Create a type that selects only id and name from User
type UserBasic = // Your code:

// 3. Create a type that excludes password from User
interface UserWithPassword {
    id: number;
    name: string;
    email: string;
    password: string;
    age: number;
}
type UserPublic = // Your code:

// 4. Create a type that makes all properties readonly
type ReadonlyUser = // Your code:
```

### Solution:

```
type OptionalUser = Partial<User>;

type UserBasic = Pick<User, "id" | "name">;

interface UserWithPassword {
    id: number;
    name: string;
    email: string;
    password: string;
    age: number;
}
type UserPublic = Omit<UserWithPassword, "password">;

type ReadonlyUser = Readonly<User>;
```

## Exercise 4.4: Custom Utility Types

**Instructions:** Create custom utility types.

```
// 1. Create a type that makes all properties nullable
type Nullable<T> = {
    // Your code:
};
```

```
// 2. Create a type that extracts only string properties
type StringKeys<T> = {
  // Your code:
}[keyof T];

// 3. Create a DeepReadonly type
type DeepReadonly<T> = {
  // Your code:
};
```

### Solution:

```
type Nullable<T> = {
  [P in keyof T]: T[P] | null;
};

type StringKeys<T> = {
  [P in keyof T]: T[P] extends string ? P : never;
}[keyof T];

type DeepReadonly<T> = {
  readonly [P in keyof T]: T[P] extends object ? DeepReadonly<T[P]> : T[P];
};

// Test
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
  address: {
    street: string;
    city: string;
    zip: string;
  };
}

type NullableUser = Nullable<User>;
type UserStringKeys = StringKeys<User>; // "name" | "email" | "street" | "city" | "zip"
type DeepReadonlyUser = DeepReadonly<User>;
```

## Day 4 Summary

---

### Key Takeaways

1. **Generics:** Reusable components with type safety
2. **Utility Types:** `Partial` , `Required` , `Readonly` , `Pick` , `Omit` , `Record` , `Exclude` , `Extract` , `NonNullable` , `ReturnType` , `Parameters`
3. **Custom Utility Types:** Create your own with mapped types
4. **Conditional Types:** `T extends U ? X : Y`
5. **Infer:** Extract types from other types
6. **Template Literal Types:** String manipulation at type level

## Quiz Yourself

1. What are generics?
2. How do you constrain a generic type?
3. What is the difference between `Pick` and `Omit` ?
4. What is a conditional type?
5. What does `infer` do?

## Day 4 Homework

Create a TypeScript file that:

1. Defines a generic `Repository<T>` class with CRUD operations
2. Uses utility types to create different views of a `User` type
3. Creates a custom utility type that makes all properties optional and nullable
4. Uses template literal types to create API endpoint types

# Day 5: TypeScript Project & Real-World Applications

---

## Learning Objectives

By the end of Day 5, you will be able to:

- Build a complete TypeScript project
- Configure TypeScript for different environments
- Use TypeScript with React
- Use TypeScript with Node.js
- Write type-safe code in real-world scenarios

## 5.1 TypeScript Configuration

### tsconfig.json Options

```
{
  "compilerOptions": {
    // Language and Environment
    "target": "ES2020", // JavaScript version
    "lib": ["ES2020", "DOM"], // Available libraries
    "jsx": "react", // JSX support

    // Modules
    "module": "commonjs", // Module system
    "moduleResolution": "node", // Module resolution
    "baseUrl": ".", // Base directory for module resolution
    "paths": { // Path aliases
      "@/*": ["./src/*"]
    },

    // Emit
    "outDir": "./dist", // Output directory
    "rootDir": "./src", // Source directory
    "sourceMap": true, // Generate source maps

    // Type Checking
    "strict": true, // Enable all strict options
    "noImplicitAny": true, // Error on implicit any
    "strictNullChecks": true, // Strict null checks
    "noUnusedLocals": true, // Error on unused locals
    "noUnusedParameters": true, // Error on unused parameters
    "noImplicitReturns": true, // Error on implicit returns
    "noFallthroughCasesInSwitch": true, // Error on fallthrough

    // Interop Constraints
    "esModuleInterop": true, // Interop with CommonJS
    "skipLibCheck": true, // Skip type checking of declaration files
    "forceConsistentCasingInFileNames": true // Case sensitivity

    // Advanced
    "resolveJsonModule": true, // Allow importing JSON
    "isolatedModules": true, // Transpile modules independently
    "noEmit": false, // Emit output
  },
  "include": ["src/**/*"], // Files to include
  "exclude": ["node_modules", "dist"] // Files to exclude
}
```

## React Component Types

```
// Functional component with TypeScript
import React, { FC, useState, useEffect } from 'react';

// Props interface
interface UserProps {
  name: string;
  age: number;
  email?: string;
}

// Functional component
const UserCard: FC<UserProps> = ({ name, age, email }) => {
  const [isActive, setIsActive] = useState(false);

  useEffect(() => {
    console.log(`${name} loaded`);
  }, [name]);

  return (
    <div className="user-card">
      <h2>{name}</h2>
      <p>Age: {age}</p>
      {email && <p>Email: {email}</p>}
      <button onClick={() => setIsActive(!isActive)}>
        {isActive ? 'Active' : 'Inactive'}
      </button>
    </div>
  );
};

// Event handlers with types
const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
  console.log(event.target.value);
};

const handleSubmit = (event: React.FormEvent<HTMLFormElement>) => {
  event.preventDefault();
  // Form submission logic
};
```

## Custom Hooks with TypeScript

```
import { useState, useEffect } from 'react';

// Custom hook with generics
function useFetch<T>(url: string): {
  data: T | null;
  loading: boolean;
```

```

    error: Error | null;
  } {
    const [data, setData] = useState<T | null>(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<Error | null>(null);

    useEffect(() => {
      const fetchData = async () => {
        try {
          const response = await fetch(url);
          const json = await response.json();
          setData(json);
        } catch (err) {
          setError(err instanceof Error ? err : new Error('Unknown error'));
        } finally {
          setLoading(false);
        }
      };

      fetchData();
    }, [url]);

    return { data, loading, error };
  }

// Usage
interface User {
  id: number;
  name: string;
  email: string;
}

function UserList() {
  const { data, loading, error } = useFetch<User[]>('/api/users');

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;

  return (
    <ul>
      {data?.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

```

## 5.2 TypeScript with Node.js

---

## Node.js TypeScript Setup

```
# Initialize project
npm init -y

# Install dependencies
npm install --save-dev typescript @types/node ts-node nodemon

# Create tsconfig.json
npx tsc --init

# Install runtime dependencies
npm install express dotenv
npm install --save-dev @types/express
```

## Express with TypeScript

```
// src/index.ts
import express, { Request, Response, NextFunction } from 'express';
import dotenv from 'dotenv';

dotenv.config();

const app = express();
const port = process.env.PORT || 3000;

// Type for request body
interface CreateUserRequest {
  name: string;
  email: string;
  age: number;
}

// Type for response
interface UserResponse {
  id: number;
  name: string;
  email: string;
  age: number;
  createdAt: Date;
}

// Middleware with types
const logger = (req: Request, res: Response, next: NextFunction) => {
  console.log(`${req.method} ${req.url}`);
  next();
};

app.use(express.json());
app.use(logger);
```

```

// GET endpoint
app.get('/api/users', (req: Request, res: Response) => {
  const users: UserResponse[] = [
    { id: 1, name: 'Alice', email: 'alice@example.com', age: 25, createdAt: new Date() },
    { id: 2, name: 'Bob', email: 'bob@example.com', age: 30, createdAt: new Date() }
  ];
  res.json(users);
});

// POST endpoint
app.post('/api/users', (req: Request<{}, {}, CreateUserRequest>, res: Response) => {
  const { name, email, age } = req.body;

  // Validation
  if (!name || !email || !age) {
    return res.status(400).json({ error: 'Missing required fields' });
  }

  const newUser: UserResponse = {
    id: Date.now(),
    name,
    email,
    age,
    createdAt: new Date()
  };

  res.status(201).json(newUser);
});

// Error handling middleware
app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});

```

## 5.3 Real-World Patterns

---

### Repository Pattern

```

// Repository interface
interface Repository<T> {
  findById(id: number): Promise<T | null>;
  findAll(): Promise<T[]>;
}

```

```
    create(item: Omit<T, 'id'>): Promise<T>;
    update(id: number, item: Partial<T>): Promise<T | null>;
    delete(id: number): Promise<boolean>;
}

// User type
interface User {
    id: number;
    name: string;
    email: string;
    age: number;
    createdAt: Date;
}

// In-memory implementation
class UserRepository implements Repository<User> {
    private users: User[] = [];
    private nextId = 1;

    async findById(id: number): Promise<User | null> {
        return this.users.find(user => user.id === id) || null;
    }

    async findAll(): Promise<User[]> {
        return this.users;
    }

    async create(item: Omit<User, 'id'>): Promise<User> {
        const newUser: User = {
            ...item,
            id: this.nextId++
        };
        this.users.push(newUser);
        return newUser;
    }

    async update(id: number, item: Partial<User>): Promise<User | null> {
        const index = this.users.findIndex(user => user.id === id);
        if (index === -1) return null;

        const updatedUser = { ...this.users[index], ...item };
        this.users[index] = updatedUser;
        return updatedUser;
    }

    async delete(id: number): Promise<boolean> {
        const index = this.users.findIndex(user => user.id === id);
        if (index === -1) return false;

        this.users.splice(index, 1);
        return true;
    }
}
```

```
}  
}
```

## Service Pattern

```
// Service class  
class UserService {  
    constructor(private repository: Repository<User>) {}  
  
    async getUser(id: number): Promise<User | null> {  
        return this.repository.findById(id);  
    }  
  
    async getAllUsers(): Promise<User[]> {  
        return this.repository.findAll();  
    }  
  
    async createUser(userData: Omit<User, 'id'>): Promise<User> {  
        // Business logic  
        if (userData.age < 18) {  
            throw new Error('User must be 18 or older');  
        }  
  
        // Email validation  
        if (!userData.email.includes('@')) {  
            throw new Error('Invalid email');  
        }  
  
        return this.repository.create(userData);  
    }  
  
    async updateUser(id: number, userData: Partial<User>): Promise<User | null> {  
        // Only allow certain fields to be updated  
        const allowedUpdates: (keyof User)[] = ['name', 'age'];  
        const filteredData: Partial<User> = {};  
  
        for (const key of allowedUpdates) {  
            if (key in userData) {  
                filteredData[key] = userData[key];  
            }  
        }  
  
        return this.repository.update(id, filteredData);  
    }  
  
    async deleteUser(id: number): Promise<boolean> {  
        return this.repository.delete(id);  
    }  
}  
  
// Usage
```

```
const userRepository = new UserRepository();
const userService = new UserService(userRepository);

// Create a user
const newUser = await userService.createUser({
  name: 'Alice',
  email: 'alice@example.com',
  age: 25,
  createdAt: new Date()
});

console.log(newUser);
```

## 5.4 TypeScript with APIs

---

### API Client with TypeScript

```
// API client
interface ApiClientOptions {
  baseUrl: string;
  timeout: number;
  headers?: Record<string, string>;
}

interface ApiResponse<T> {
  data: T;
  status: number;
  success: boolean;
}

class ApiClient {
  private baseUrl: string;
  private timeout: number;
  private headers: Record<string, string>;

  constructor(options: ApiClientOptions) {
    this.baseUrl = options.baseUrl;
    this.timeout = options.timeout;
    this.headers = options.headers || {};
  }

  private async request<T>(url: string, options: RequestInit): Promise<ApiResponse<T> > {
    const response = await fetch(`${this.baseUrl}${url}`, {
      ...options,
      headers: {
        ...this.headers,
        ...options.headers
      }
    })
  }
```

```
    });

    const data = await response.json();

    return {
      data,
      status: response.status,
      success: response.ok
    };
  }

  async get<T>(url: string): Promise<ApiResponse<T>> {
    return this.request<T>(url, { method: 'GET' });
  }

  async post<T>(url: string, body: unknown): Promise<ApiResponse<T>> {
    return this.request<T>(url, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body)
    });
  }

  async put<T>(url: string, body: unknown): Promise<ApiResponse<T>> {
    return this.request<T>(url, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body)
    });
  }

  async delete<T>(url: string): Promise<ApiResponse<T>> {
    return this.request<T>(url, { method: 'DELETE' });
  }
}

// Usage
interface User {
  id: number;
  name: string;
  email: string;
}

interface CreateUserDto {
  name: string;
  email: string;
  password: string;
}

const apiClient = new ApiClient({
  baseUrl: 'https://api.example.com',
  timeout: 5000,
  headers: {
```

```

    'Authorization': 'Bearer token'
  }
});

// GET users
const response = await apiClient.get<User[]>('/users');
if (response.success) {
  console.log('Users:', response.data);
}

// Create user
const newUser = await apiClient.post<User>('/users', {
  name: 'Alice',
  email: 'alice@example.com',
  password: 'securePassword123'
},
`

```

## 5.5 Advanced TypeScript Tips

### Branded Types

```

// Branded types for type safety
type UserId = string & { __brand: 'UserId' };
type Email = string & { __brand: 'Email' };
type OrderId = string & { __brand: 'OrderId' };

function createUserId(id: string): UserId {
  return id as UserId;
}

function getOrder(id: OrderId) {
  // This function expects an OrderId, not a generic string
}

const userId = createUserId('user-123');
// getOrder(userId); // Error: Argument of type 'UserId' is not assignable to paramet
`

```

### Discriminated Unions

```

// Discriminated union for different states
type State =
  | { status: 'loading' }
  | { status: 'success'; data: User[] }
  | { status: 'error'; error: string }
  | { status: 'idle' };
`

```

```
function renderState(state: State): string {
  switch (state.status) {
    case 'loading': return 'Loading...';
    case 'success': return `Loaded ${state.data.length} users`;
    case 'error': return `Error: ${state.error}`;
    case 'idle': return 'Ready to load';
  }
}

const state: State = { status: 'success', data: [{ id: 1, name: 'Alice', email: '' }]
  // ... (omitted) ...
}
```

## Type Safety with Assertions

```
// Runtime type checking
function isUser(value: unknown): value is User {
  return typeof value === 'object' &&
    value !== null &&
    'id' in value &&
    'name' in value &&
    'email' in value;
}

// Use in function
function processUser(value: unknown): void {
  if (isUser(value)) {
    console.log(`User: ${value.name}`);
  } else {
    console.log('Invalid user object');
  }
}
```

## Day 5 Exercises

---

### Exercise 5.1: Complete TypeScript Project

**Instructions:** Build a complete TypeScript project.

```
// 1. Create a TodoItem type
type TodoItem = {
  // Your code:
};

// 2. Create a TodoService class
class TodoService {
```

```

    // Your code:
}

// 3. Create a TodoController class
class TodoController {
    // Your code:
}

// 4. Create a simple CLI interface
// Your code:

```

## Solution:

```

type TodoItem = {
    id: number;
    title: string;
    completed: boolean;
    createdAt: Date;
    updatedAt?: Date;
};

class TodoService {
    private todos: TodoItem[] = [];
    private nextId = 1;

    create(title: string): TodoItem {
        const newTodo: TodoItem = {
            id: this.nextId++,
            title,
            completed: false,
            createdAt: new Date()
        };
        this.todos.push(newTodo);
        return newTodo;
    }

    findAll(): TodoItem[] {
        return this.todos;
    }

    findById(id: number): TodoItem | undefined {
        return this.todos.find(todo => todo.id === id);
    }

    update(id: number, data: Partial<Omit<TodoItem, 'id' | 'createdAt'>>): TodoItem |
        const todo = this.findById(id);
        if (!todo) return undefined;

        const updatedTodo = {
            ...todo,
            ...data,

```

```

        updatedAt: new Date()
    };

    const index = this.todos.indexOf(todo);
    this.todos[index] = updatedTodo;
    return updatedTodo;
}

delete(id: number): boolean {
    const index = this.todos.findIndex(todo => todo.id === id);
    if (index === -1) return false;

    this.todos.splice(index, 1);
    return true;
}

toggleComplete(id: number): TodoItem | undefined {
    const todo = this.findById(id);
    if (!todo) return undefined;

    return this.update(id, { completed: !todo.completed });
}
}

class TodoController {
    constructor(private service: TodoService) {}

    addTodo(title: string): void {
        const todo = this.service.create(title);
        console.log(`✅ Added: "${todo.title}" (ID: ${todo.id})`);
    }

    listTodos(): void {
        const todos = this.service.findAll();
        if (todos.length === 0) {
            console.log('📭 No todos');
            return;
        }

        console.log('📭 Your Todos:');
        todos.forEach(todo => {
            const status = todo.completed ? '✅' : '❌';
            console.log(`${status} [${todo.id}] ${todo.title}`);
        });
    }

    completeTodo(id: number): void {
        const todo = this.service.toggleComplete(id);
        if (todo) {
            console.log(`✅ Todo ${id} marked as ${todo.completed ? 'completed' : 'incomplete'}`);
        } else {
            console.log(`❌ Todo with ID ${id} not found`);
        }
    }
}

```

```

    }

    deleteTodo(id: number): void {
        const success = this.service.delete(id);
        if (success) {
            console.log(`🗑️ Todo ${id} deleted`);
        } else {
            console.log(`❌ Todo with ID ${id} not found`);
        }
    }
}

// CLI Interface
import readline from 'readline';

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});

const service = new TodoService();
const controller = new TodoController(service);

function showHelp(): void {
    console.log(`
Commands:
add "title"      - Add a new todo
list             - List all todos
complete <id>    - Mark todo as complete/incomplete
delete <id>      - Delete a todo
help            - Show this help
exit            - Exit the application
`);
}

console.log(`📝 TypeScript Todo App`);
showHelp();

rl.on('line', (input: string) => {
    const [command, ...args] = input.trim().split(' ');
    const arg = args.join(' ');

    switch (command.toLowerCase()) {
        case 'add':
            if (arg) {
                controller.addTodo(arg);
            } else {
                console.log(`❌ Please provide a title: add "My todo"`);
            }
            break;

        case 'list':
            controller.listTodos();
    }
});

```

```

        break;

    case 'complete':
        const completeId = parseInt(arg);
        if (!isNaN(completeId)) {
            controller.completeTodo(completeId);
        } else {
            console.log('❌ Please provide a valid ID: complete 1');
        }
        break;

    case 'delete':
        const deleteId = parseInt(arg);
        if (!isNaN(deleteId)) {
            controller.deleteTodo(deleteId);
        } else {
            console.log('❌ Please provide a valid ID: delete 1');
        }
        break;

    case 'help':
        showHelp();
        break;

    case 'exit':
    case 'quit':
        console.log('👋 Goodbye!');
        rl.close();
        process.exit(0);

    default:
        console.log(`❌ Unknown command: "${command}". Type "help" for available c
    }
});

```

## Day 5 Summary

---

### Key Takeaways

1. **tsconfig.json**: Configure TypeScript for your project
2. **React with TypeScript**: Props, state, events, custom hooks
3. **Node.js with TypeScript**: Express, middleware, error handling
4. **Repository Pattern**: Data access abstraction
5. **Service Pattern**: Business logic abstraction
6. **API Clients**: Type-safe HTTP clients

## 7. Advanced Patterns: Branded types, discriminated unions, type guards

### Quiz Yourself

1. What is the purpose of tsconfig.json?
2. How do you define props in a React component with TypeScript?
3. What is the Repository pattern?
4. What are branded types used for?
5. What is a discriminated union?

### Final Project

Build a complete TypeScript application of your choice:

- A REST API with Express and TypeScript
- A CLI tool with TypeScript
- A React application with TypeScript
- A full-stack application with TypeScript

## Final Assessment

---

### TypeScript Mastery Test

---

#### Section 1: Basic Types

**Instructions:** Annotate the following variables with the correct TypeScript types.

```
let name = "Alice";           // Type: _____
let age = 25;                 // Type: _____
let isActive = true;         // Type: _____
let scores = [85, 90, 78];    // Type: _____
let person = { name: "Bob", age: 30 }; // Type: _____
```

#### Section 2: Functions

**Instructions:** Write the TypeScript function with proper types.

```
// Function that takes a string and returns its length
function getStringLength(text: string): number {
  // Your code:
```

```
}

// Function that takes two numbers and returns their sum
function addNumbers(a: number, b: number): number {
  // Your code:
}

// Function that takes an array of numbers and returns the average
function calculateAverage(numbers: number[]): number {
  // Your code:
}
```

## Section 3: Interfaces and Types

**Instructions:** Define the following interfaces and types.

```
// 1. Define a Product interface with id, name, price, and optional description
interface Product {
  // Your code:
}

// 2. Define a type alias for a function that takes a number and returns a boolean
type Predicate = // Your code:

// 3. Define a union type for HTTP methods
type HttpMethod = // Your code:
```

## Section 4: Generics

**Instructions:** Write generic functions and classes.

```
// 1. Generic function that returns the first element of an array
function firstElement<T>(array: T[]): T | undefined {
  // Your code:
}

// 2. Generic class for a KeyValuePair
class KeyValuePair<K, V> {
  // Your code:
}

// 3. Generic function that filters an array
function filterArray<T>(array: T[], predicate: (item: T) => boolean): T[] {
  // Your code:
}
```

## Section 5: Utility Types

**Instructions:** Use utility types to transform these types.

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  age: number;
}

// 1. Make all properties optional
type OptionalUser = // Your code:

// 2. Make all properties readonly
type ReadonlyUser = // Your code:

// 3. Pick only id and name
type UserBasic = // Your code:

// 4. Omit password
type UserPublic = // Your code:
```

## Section 6: Classes

**Instructions:** Create a class with proper TypeScript.

```
// Create a BankAccount class with:
// - Properties: accountNumber (readonly), balance (private), owner (protected)
// - Methods: deposit, withdraw, getBalance
// - Constructor: initializes all properties

class BankAccount {
  // Your code:
}

// Create a SavingsAccount class that extends BankAccount
// - Adds interestRate property
// - Adds addInterest method

class SavingsAccount extends BankAccount {
  // Your code:
}
```

## Section 7: Real-World Application

**Instructions:** Write a complete TypeScript application.

```
// Build a simple task manager with:  
// 1. A Task interface or type  
// 2. A TaskService class with CRUD operations  
// 3. A TaskController class with CLI commands  
// 4. A simple CLI interface
```

```
// Your code here:
```

## Assessment Answer Key

---

### Section 1: Basic Types

```
let name: string = "Alice";  
let age: number = 25;  
let isActive: boolean = true;  
let scores: number[] = [85, 90, 78];  
let person: { name: string; age: number; } = { name: "Bob", age: 30 };
```

### Section 2: Functions

```
function getStringLength(text: string): number {  
    return text.length;  
}  
  
function addNumbers(a: number, b: number): number {  
    return a + b;  
}  
  
function calculateAverage(numbers: number[]): number {  
    const sum = numbers.reduce((total, num) => total + num, 0);  
    return sum / numbers.length;  
}
```

### Section 3: Interfaces and Types

```
interface Product {  
    id: number;  
    name: string;  
    price: number;
```

```
    description?: string;
  }

  type Predicate = (value: number) => boolean;

  type HttpMethod = "GET" | "POST" | "PUT" | "DELETE";
```

## Section 4: Generics

```
function firstElement<T>(array: T[]): T | undefined {
  return array[0];
}

class KeyValuePair<K, V> {
  constructor(private key: K, private value: V) {}

  getKey(): K {
    return this.key;
  }

  getValue(): V {
    return this.value;
  }
}

function filterArray<T>(array: T[], predicate: (item: T) => boolean): T[] {
  return array.filter(predicate);
}
```

## Section 5: Utility Types

```
type OptionalUser = Partial<User>;
type ReadonlyUser = Readonly<User>;
type UserBasic = Pick<User, "id" | "name">;
type UserPublic = Omit<User, "password">;
```

## Section 6: Classes

```
class BankAccount {
  public readonly accountNumber: string;
  private balance: number;
  protected owner: string;

  constructor(accountNumber: string, initialBalance: number, owner: string) {
    this.accountNumber = accountNumber;
    this.balance = initialBalance;
    this.owner = owner;
  }
}
```

```
}

deposit(amount: number): void {
    if (amount <= 0) throw new Error("Amount must be positive");
    this.balance += amount;
}

withdraw(amount: number): boolean {
    if (amount <= 0 || amount > this.balance) return false;
    this.balance -= amount;
    return true;
}

getBalance(): number {
    return this.balance;
}
}

class SavingsAccount extends BankAccount {
    private interestRate: number;

    constructor(accountNumber: string, initialBalance: number, owner: string, interest
        super(accountNumber, initialBalance, owner);
        this.interestRate = interestRate;
    }

    addInterest(): void {
        const interest = this.getBalance() * this.interestRate;
        this.deposit(interest);
    }
},
```

# TypeScript Quick Reference

## Basic Types

| Type      | Example      |
|-----------|--------------|
| string    | "Hello"      |
| number    | 25 , 3.14    |
| boolean   | true , false |
| null      | null         |
| undefined | undefined    |

| Type    | Example                                        |
|---------|------------------------------------------------|
| any     | let anything: any = 5;                         |
| unknown | let value: unknown = 5;                        |
| never   | function error(): never { throw new Error(); } |
| void    | function log(): void { console.log("Hello"); } |

## Arrays and Tuples

---

```
let numbers: number[] = [1, 2, 3];
let strings: Array<string> = ["a", "b", "c"];
let tuple: [string, number] = ["Alice", 25];
```

## Functions

---

```
function add(a: number, b: number): number {
    return a + b;
}

const multiply = (a: number, b: number): number => a * b;

const greet: (name: string) => string = (name) => `Hello, ${name}`;
```

## Interfaces

---

```
interface Person {
    name: string;
    age: number;
    email?: string;
}

interface Employee extends Person {
    employeeId: string;
}
```

## Type Aliases

---

```
type ID = string | number;
type Status = "pending" | "success" | "error";
```

```
type User = { id: ID; name: string; };
```

# Generics

```
function identity<T>(value: T): T {
  return value;
}

class Box<T> {
  constructor(private value: T) {}
  getValue(): T { return this.value; }
}
```

# Utility Types

| Type           | Description                                     |
|----------------|-------------------------------------------------|
| Partial<T>     | Makes all properties optional                   |
| Required<T>    | Makes all properties required                   |
| Readonly<T>    | Makes all properties readonly                   |
| Pick<T, K>     | Selects a subset of properties                  |
| Omit<T, K>     | Excludes a subset of properties                 |
| Record<K, T>   | Creates an object type with keys K and values T |
| Exclude<T, U>  | Excludes types from T that are in U             |
| Extract<T, U>  | Extracts types from T that are in U             |
| NonNullable<T> | Excludes null and undefined                     |
| ReturnType<T>  | Gets the return type of a function              |
| Parameters<T>  | Gets the parameter types of a function          |

# Type Guards

```
// typeof guard
if (typeof value === "string") {
  // value is string
}

// instanceof guard
```

```
if (value instanceof Array) {  
    // value is array  
}  
  
// Custom type guard  
function isNumber(value: unknown): value is number {  
    return typeof value === "number";  
}
```

## Classes

---

```
class Person {  
    public name: string;  
    private age: number;  
    protected role: string;  
  
    constructor(name: string, age: number, role: string) {  
        this.name = name;  
        this.age = age;  
        this.role = role;  
    }  
  
    greet(): string {  
        return `Hello, I'm ${this.name}`;  
    }  
}
```

## TypeScript Glossary

---

### A

---

**Abstract Class:** A class that cannot be instantiated directly. Used as a base class for other classes.

**Access Modifier:** Keywords that control visibility of class members: `public` , `private` , `protected` .

**Alias:** A type alias gives a name to a type.

**Any:** A type that disables type checking. Use with caution.

### B

---

**Boolean:** A primitive type representing true/false values.

**Branded Type:** A type with a unique marker to prevent misuse.

## C

---

**Class:** A blueprint for creating objects with properties and methods.

**Conditional Type:** A type that selects different types based on a condition.

**Const Assertion:** `as const` to infer literal types.

**Constructor:** A special method in a class that initializes new instances.

## D

---

**Discriminated Union:** A union type with a common property to distinguish members.

## E

---

**Exhaustive Checking:** Using `never` to ensure all union cases are handled.

**Expression:** Code that produces a value.

**Extends:** Used for inheritance ( `extends` ) and type constraints ( `T extends U` ).

## G

---

**Generic:** A type parameter that makes components reusable.

**Getter:** A method that accesses a property value.

## I

---

**Index Signature:** A way to define dynamic property names.

**Infer:** Extracts types within conditional types.

**Interface:** Defines the shape of an object.

**Intersection Type:** Combines multiple types with `&` .

## L

---

**Literal Type:** A specific value as a type ( `"success" | "error"` ).

## M

---

**Mapped Type:** A type that transforms properties of another type.

**Method:** A function defined within a class.

## N

---

**Never:** A type representing values that never occur.

**Null:** A primitive type representing empty value.

**Number:** A primitive type for numeric values.

## O

---

**Omit:** Utility type that excludes specific properties.

**Optional:** A property that may be undefined ( `property?: type` ).

## P

---

**Parameter:** Input to a function.

**Partial:** Utility type that makes all properties optional.

**Pick:** Utility type that selects specific properties.

**Polymorphism:** The ability to use objects of different types through a common interface.

**Private:** Access modifier that restricts access to the class only.

**Protected:** Access modifier that restricts access to the class and subclasses.

**Public:** Access modifier that allows access anywhere (default).

## R

---

**ReadOnly:** A modifier that prevents property reassignment.

**Record:** Utility type for creating object types.

**Return Type:** The type of value a function returns.

## S

---

**Setter:** A method that sets a property value.

**String:** A primitive type for text data.

**Super:** A keyword to access the parent class.

## T

---

**Tuple:** A fixed-length array with specific types for each position.

**Type:** A classification of data.

**Type Alias:** A name for a type.

**Type Annotation:** Explicitly specifying a type ( `: type` ).

**Type Assertion:** Telling TypeScript a value's type ( `as Type` ).

**Type Guard:** A condition that narrows a type.

**Type Inference:** TypeScript automatically determining a type.

## U

---

**Undefined:** A primitive type representing not defined value.

**Union Type:** A type that can be one of several types ( `type | type` ).

**Utility Type:** Built-in types for common transformations.

## V

---

**Void:** A type indicating no return value.

## Course Completion Certificate

---

This certifies that

---

has successfully completed the

**5-Day TypeScript Programming Foundations Course**

covering all fundamental concepts including basic types, functions, interfaces, classes, generics, utility types, and real-world TypeScript applications.

**Date:** \_\_\_\_\_

**Instructor:** \_\_\_\_\_

**Signature:** \_\_\_\_\_